

Fuzzy Early Warning System

Temporal Context Tab Overview

*Focused on Tab 3 only: membership functions,
snapshot scoring, and temporal adjustment*

Generated: 2026-04-09 14:14

Table of Contents

1. What the Temporal Context tab does
2. How the input membership functions are made
3. How one observation becomes a snapshot score
4. How the tab builds a timeline
5. EWMA memory, trend detection, and adjustment
6. Final total score and risk bucket
7. Parameters shown in Tab 3

1. What the Temporal Context tab does

The Temporal Context tab is the third tab in the Streamlit app. It starts with the same per-observation fuzzy scoring pipeline as the Type-1 tab, but then carries information forward through time. In short, Tab 3 asks how abnormal the newest observation is, how much recent history matters, and whether the vital appears to be worsening.

The full Tab 3 pipeline is:

```
Raw observation
-> clamp to membership-table range
-> evaluate sigmoid input membership functions
-> collapse labels into 4 concern levels
-> centroid defuzzify each vital to a score in [0, 3]
-> aggregate 6 per-vital scores to a snapshot total in [0, 18]
-> store the observation in a timeline
-> EWMA smooth each vital across time
-> detect worsening trend in the raw per-vital scores
-> push worsening vitals upward with a sigmoid trend factor
-> aggregate adjusted per-vital scores into the final temporal total
```

2. How the input membership functions are made

Tab 3 uses the generated sigmoid membership tables stored in `generated_membership_data/sigmoid/`. Each vital loads a CSV lookup table with one Value column and one column for each linguistic label.

That means the membership functions are pre-generated smooth sigmoid curves sampled over a grid and saved into CSV files. The app reads these lookup tables directly instead of rebuilding the curves during each run. Compared with the older hard-edged category tables in `data/membership_function_plots/csv_data/`, the generated sigmoid tables let neighbouring labels overlap smoothly.

The missing construction step is that these smooth curves were not meant to be arbitrary sigmoids. In this project, they were described as being built from clinician-defined thresholds together with the clinicians' spread or variance, then converted into smooth transitions using Gaussian cumulative distribution functions (CDFs).

CDF-based construction idea

Conceptually, each threshold between adjacent clinical categories is treated as the centre of a smooth transition rather than a hard cut-off. A Gaussian CDF is then used to turn that boundary into an S-shaped curve. The spread parameter sigma controls how gradual the transition is: smaller sigma gives a steeper boundary, while larger sigma gives a softer one.

$\text{Phi}(x; \mu, \sigma) = \text{normal CDF centred at } \mu \text{ with spread } \sigma$

For edge categories, a single CDF can define the membership shape. For example, a low-extreme category can be represented by a decreasing CDF, while a high-extreme category can be represented by an increasing CDF.

```
low-side category    ~ 1 - Phi(x; boundary, sigma)
high-side category   ~ Phi(x; boundary, sigma)
```

For middle categories, two neighbouring CDF transitions are combined so the membership rises from one boundary and falls at the next, producing the smooth hill-shaped regions seen in the generated tables.

```
middle category ~ Phi(x; left_boundary, sigma_left)
                - Phi(x; right_boundary, sigma_right)
```

The practical outcome is that the old sharp threshold system becomes a set of overlapping smooth curves whose transitions are controlled by clinician uncertainty rather than by arbitrary straight-line corners.

A useful property of this construction is the partition-of-unity behaviour described elsewhere in the project: across the input range, the category memberships are designed to sum to approximately 1 at each x-value. That is why a value can partially belong to two neighbouring labels without the total membership becoming unstable.

So, in teaching terms: the generated CSVs in Tab 3 are the final sampled result of a CDF-based smoothing process. The app itself only loads and interpolates those saved curves, but the reason they look sigmoid is that they were originally constructed from Gaussian CDF transitions.

Membership families

Vital	Family	Labels
-------	--------	--------

Heart rate	Below severe	below severe, below moderate, below mild, no concern, above mild, above moderate, above severe
Systolic BP	Below severe	below severe, below moderate, below mild, no concern, above mild, above moderate, above severe
Temperature	Below severe	below severe, below moderate, below mild, no concern, above mild, above moderate, above severe
Respiratory rate	Below severe	below severe, below moderate, below mild, no concern, above mild, above moderate, above severe
SpO2	Downward	Below severe, below moderate, below mild, no concern
FiO2	Upward	No concern, above mild, above moderate, above severe

Why the families differ: heart rate, blood pressure, temperature, and respiratory rate can be dangerous when too low or too high, so they use a symmetric 7-label design. SpO2 is mainly dangerous when it falls, so its labels only move downward. FiO2 is mainly concerning when it rises, because a higher oxygen requirement suggests greater support.

In the CDF view, that means the 7-label vitals use multiple adjacent Gaussian-CDF transitions on both sides of a central normal region, whereas SpO2 and FiO2 only need one-sided transition stacks because concern mainly develops in one direction.

Lookup and interpolation

When the user enters a value, the app finds the two nearest rows in the CSV and linearly interpolates between them. This turns a discrete lookup table into a continuous membership function.

$$t = (v - k_i) / (k_{(i+1)} - k_i)$$

$$\mu(v) = \mu(k_i) * (1 - t) + \mu(k_{(i+1)}) * t$$

Boundary handling in Tab 3:

- Before scoring, every observation is clamped to the minimum and maximum Value in the relevant CSV.
- If the user enters a value outside the available grid, the model uses the nearest edge of the generated table.
- Because neighbouring sigmoid labels overlap, one observation can partially belong to multiple labels at once.

3. How one observation becomes a snapshot score

Tab 3 first computes an ordinary snapshot fuzzy score for the newest observation. Temporal logic is added only after that. The snapshot stage has three core parts: label firing, concern-level consolidation, and centroid defuzzification.

Stage A: evaluate the input labels

Each vital is evaluated against its sigmoid input membership functions. The result is a set of firing strengths between 0 and 1.

Stage B: collapse many labels into 4 concern levels

The app maps all input labels into four output concern levels: No concern, Mild concern, Moderate concern, and Severe concern. The mapping uses max-aggregation, so both low-severe and high-severe labels feed into the same Severe concern output.

```
for each input label with strength mu:
    if 'severe' in label: Severe = max(Severe, mu)
    if 'moderate' in label: Moderate = max(Moderate, mu)
    if 'mild' in label: Mild = max(Mild, mu)
    if 'no concern' in label: No = max(No, mu)
```

Stage C: defuzzify to a per-vital score in [0, 3]

The output universe for each vital is 0 to 3. Four trapezoidal output membership functions represent No, Mild, Moderate, and Severe concern.

Concern	a	b	c	d
No concern	-0.5	0	0	0.75
Mild concern	0.25	1	1	1.75
Moderate concern	1.25	2	2	2.75
Severe concern	2.25	3	3	3.5

The app evaluates these outputs on 301 points from 0.00 to 3.00, clips each set by its firing strength, takes the pointwise maximum, and then computes the centroid.

$$\text{score} = \text{SUM}(x_i * \mu_{\text{agg}}(x_i)) / \text{SUM}(\mu_{\text{agg}}(x_i))$$

Two implementation details matter:

- Firings below 0.05 are zeroed so tiny edge overlaps do not create noise.
- If only the No concern set remains active, the app returns an exact score of 0.0.

After this has been done for all six vitals, the per-vital scores are aggregated into the snapshot total on the 0 to 18 scale.

4. How the tab builds a timeline

Each time the user clicks Add observation, Tab 3 stores the raw values, the clamped model values, the six per-vital fuzzy scores, and the snapshot totals. The user also chooses an interval size and unit, so each new row gets a time in minutes from the start of the sequence.

Every stored row therefore has both inputs and model outputs:

- Clinical inputs: heart rate, blood pressure, temperature, respiratory rate, SpO2, and FiO2.
- Model outputs: six per-vital fuzzy scores, snapshot fuzzy total, and NEWS-2 snapshot total.

That sequence becomes the input to the temporal adjustment stage. The newest observation is always interpreted against the whole timeline currently built inside the tab.

5. EWMA memory, trend detection, and adjustment

Step 1: EWMA memory

For each vital independently, Tab 3 applies an exponentially weighted moving average to the raw per-vital fuzzy scores. This is the memory part of the temporal builder.

```
EWMA[0] = raw[0]
EWMA[t] = alpha * raw[t] + (1 - alpha) * EWMA[t-1]
```

The app then clamps the smoothed value upward so temporal context cannot hide a suddenly bad current observation.

```
clamped[t] = max(EWMA[t], raw[t])
```

Step 2: estimate worsening or improvement

The trend is computed from the raw per-vital scores, not from the EWMA scores. For the latest observation, the app looks back over the selected window, converts times to hours since the first point in that window, and fits a simple ordinary least squares slope.

```
slope = SUM((t_i - t_mean)(s_i - s_mean)) / SUM((t_i - t_mean)^2)
```

Interpretation:

- Positive slope means the vital is worsening.
- Negative slope means the vital is improving.
- If fewer than two observations are in the window, slope is set to 0.

Step 3: convert worsening into a bounded trend factor

Only positive slopes trigger an upward adjustment. Tab 3 uses a sigmoid transform so the effect is smooth and bounded.

```
if slope > 0:
    trend_factor = 2 / (1 + exp(-beta * slope)) - 1
else:
    trend_factor = 0
```

Stable or improving vitals are therefore not penalised. When the slope is positive, the factor lies between 0 and 1. Higher beta makes the response steeper, so smaller positive slopes lead to larger upward pushes.

Step 4: push the score toward the maximum

```
adjusted[t] = clamped[t] + trend_factor * (3.0 - clamped[t])
adjusted[t] = clip(adjusted[t], 0.0, 3.0)
```

The term $(3.0 - \text{clamped}[t])$ is the remaining headroom to the maximum. This means a vital already near 3 can only move a little further, while a mid-range vital has more room to be pushed upward.

6. Final total score and risk bucket

After all six vitals have an EWMA score and an adjusted score, Tab 3 forms three headline totals: the latest snapshot total, the EWMA total, and the trend-adjusted total.

The totals use the same gamma rule for both the EWMA stage and the final trend-adjusted stage.

```
additive_total = SUM(per_vital_scores)
max_based_total = (18 / 3) * max(per_vital_scores)
total = (1 - gamma) * max_based_total + gamma * additive_total
```

How to read gamma:

- gamma = 1.0 keeps the total approximately additive.
- gamma = 0.0 lets the single worst vital dominate the overall total.
- Intermediate gamma values blend both ideas.

The final displayed total is clipped to the 0 to 18 range and mapped to the same risk buckets used elsewhere in the app.

Score range	Bucket	Meaning
0 to < 4	Low	Limited current or historical concern
4 to < 8	Moderate	Clear abnormality or mild accumulation
8 to < 12	High	Substantial concern across one or more vitals
12 to 18	Critical	Severe or strongly worsening concern

7. Parameters shown in Tab 3

Parameter	UI range	Default	Role
alpha	0.05 to 1.0	0.7	EWMA smoothing strength; higher alpha gives more weight to the latest value
beta	0.1 to 10.0	2.0	Controls how sharply a positive slope activates the sigmoid trend factor
Look-back window	1 to 72 h	24 h	Time window used when fitting the raw-score trend slope
gamma	0.0 to 1.0	1.0	Blends additive total with worst-vital dominance

The key learning idea is that Tab 3 is layered. The membership functions convert vitals into fuzzy scores. The timeline stores those scores. EWMA gives memory. The raw-score slope detects whether the patient is getting worse. The sigmoid trend factor keeps that adjustment smooth and bounded. Gamma then decides whether the total behaves more like a sum of all vitals or more like a worst-vital alarm.